Complete ReactJS Made Simple

Every topic explained with a picture you will never forget
+ a working code example

How to use these notes

1. Read the **story** - it is the picture that sticks.
2. Type the **code** in a Vite project (see topic 2). Never copy-paste.
3. Say the **REMEMBER** line out loud.
4. Next day, rebuild the example from memory.

Before React, know these JavaScript tools

let/const, arrow functions, template literals, destructuring, spread (...), map / filter / reduce, ES modules (import / export), promises and async/await. If any feel shaky, revise them first - React is mostly just JavaScript.

Contents

PART 1 - REACT BASICS

1. What is React and Why Use It? | 2. Setting Up with Vite | 3. JSX | 4. Components | 5. Props: Passing Data Down | 6. children Prop and Fragments | 7. Rendering Lists and Keys | 8. Conditional Rendering | 9. Handling Events | 10. Styling in React

PART 2 - STATE AND HOOKS (THE HEART OF REACT)

11. useState: Giving a Component Memory | 12. State Rules: Never Mutate, Always Replace | 13. Lifting State Up | 14. Controlled Inputs and Forms | 15. useEffect: Doing Things After Render | 16. useEffect Cleanup | 17. useRef: A Box That Does Not Redraw | 18. useContext: Skipping the Prop Chain | 19. useReducer: State with Rules | 20. useMemo: Remember an Expensive Result | 21. useCallback and React.memo | 22. Custom Hooks: Your Own Reusable Logic | 23. Rules of Hooks and Other Useful Hooks

PART 3 - BUILDING REAL APPS

24. Fetching Data from an API | 25. React Router: Multiple Pages | 26. Router: Params, Navigation, Nested and Protected Routes | 27. Global State: Redux Toolkit and Zustand | 28. Forms with React Hook Form | 29. Error Boundaries | 30. Portals and Modals | 31. Lazy Loading and Suspense | 32. Patterns: Composition, HOC and Render Props | 33. Class Components (Legacy)

PART 4 - MODERN AND ADVANCED REACT

34. Performance: Why Components Re-render | 35. React 19 Features | 36. Server Components and Next.js | 37. TypeScript with React | 38. Testing React Components | 39. Build and Deploy | 40. Folder Structure and Best Practices | 41. Common Mistakes to Avoid

FINAL

Project: To-Do App | One-page Cheat Sheet | Learning Roadmap

PART 1 - REACT BASICS

1. What is React and Why Use It?

React is a JavaScript library for building user interfaces out of **Lego bricks** called **components**. You describe WHAT the screen should look like for the current data, and React updates the page for you (**declarative**). It keeps a light copy of the page (the **Virtual DOM**), compares old vs new, and changes only what differs - fast and tidy.

```
// Without React (imperative): you give every step
const p = document.createElement("p");
p.textContent = "Count: " + count;
document.body.appendChild(p);

// With React (declarative): you describe the result
function Counter({ count }) {
  return <p>Count: {count}</p>;
}
// When count changes, React updates only that text for you.
```

REMEMBER: React = components (Lego bricks) + state. You describe the UI, React updates it.

2. Setting Up with Vite

Vite is the modern, fast tool that creates a ready React project. You need Node.js installed first.

```
# In the terminal:
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev          # open http://localhost:5173

# Folder map
# index.html         the single page
# src/main.jsx       starts React (the entry point)
# src/App.jsx        your first component
# package.json       list of packages and scripts

// src/main.jsx
import { createRoot } from "react-dom/client";
import App from "./App";
createRoot(document.getElementById("root")).render(<App />);
```

REMEMBER: npm create vite, npm install, npm run dev. main.jsx plugs App into #root.

3. JSX

JSX looks like HTML inside JavaScript. It is **sugar** that React turns into function calls. Rules: return ONE parent, close every tag, use **className** not class, camelCase attributes, and put JavaScript inside **{ }**.

```
function Profile() {
  const name = "Asha";
  const age = 20;
  const style = { color: "tomato", fontSize: "20px" };

  return (
    <div className="card">                  {/* className, not class */}
      <h1 style={style}>Hello, {name}!</h1> {/* style takes an object */}
      <p>Next year: {age + 1}</p>           {/* any expression in { } */}
      <img src="pic.png" alt="me" />        {/* self-close tags */}
      <label htmlFor="n">Name</label>       {/* htmlFor, not for */}
      <input id="n" tabIndex={0} />
    </div>
  );
}
```

REMEMBER: JSX = HTML + { JavaScript }. One parent, className, close every tag.

4. Components

A component is a **function that returns JSX**, like a cookie cutter you can use many times. Names must start with a **Capital letter**. Build big screens by nesting small components.

```
function Header() {
  return <h1>My Shop</h1>;
}

function Footer() {
  return <p>Copyright 2026</p>;
}

function App() {
  return (
    <>
      <Header />
      <main>Welcome!</main>
      <Footer />
    </>
  );
}
export default App;      // other files: import App from "./App"
```

REMEMBER: Component = Capitalized function returning JSX. Use it like a tag.

5. Props: Passing Data Down

Props are the **arguments** of a component, like ingredients handed to a chef. They flow **down** from parent to child and are **read-only**.

```
function Card({ name, price = 0, onBuy }) {   // destructure + default
  return (
    <div>
      <h3>{name}</h3>
      <p>Rs {price}</p>
      <button onClick={onBuy}>Buy</button>
    </div>
  );
}

function App() {
  return (
    <>
      <Card name="Shoes" price={999} onBuy={() => alert("Bought!")} />
      <Card name="Cap" />
    </>
  );
}
// Strings in quotes, everything else in { }. Never change props inside Card.
```

REMEMBER: Props flow down, are read-only. Functions can be props too.

6. children Prop and Fragments

children is whatever you put **between** the opening and closing tags, like a gift box with a hole for anything. A **Fragment** <> </> groups elements without adding an extra div.

```
function Box({ title, children }) {
  return (
    <section className="box">
      <h2>{title}</h2>
      {children}
    </section>
  );
}

function App() {
  return (
    <Box title="News">
      <p>React is fun.</p>
      <button>Read more</button>
    </Box>
  );
}
// <> ... </> is shorthand for <React.Fragment> ... </React.Fragment>
```

REMEMBER: children = the content between tags. Fragment = invisible wrapper.

7. Rendering Lists and Keys

Use **map** to turn an array into elements. Each item needs a unique **key**, like a roll number, so React can track who is who when the list changes.

```
const fruits = [
  { id: 1, name: "Apple" },
  { id: 2, name: "Mango" },
  { id: 3, name: "Grapes" },
];

function FruitList() {
  return (
    <ul>
      {fruits.map(f => (
        <li key={f.id}>{f.name}</li>   // key on the outer element
      ))}
    </ul>
  );
}
// Use a stable id as key. Avoid the array index if items can move or delete.
```

REMEMBER: list.map() + unique key. Prefer an id over the index.

8. Conditional Rendering

Show different things for different situations, like a traffic light for your UI.

```
function Greeting({ user, items, loading }) {
  if (loading) return <p>Loading...</p>;          // early return

  return (
    <div>
      {user ? <h1>Hi {user}</h1> : <h1>Please log in</h1>}   {/* ternary */}
      {items.length > 0 && <p>You have {items.length} items</p>}   {/* && */}
      {user && <button>Logout</button>}
    </div>
  );
}
// Careful: {0 && <p/>} prints 0! Write {count > 0 && ...} instead.
```

REMEMBER: if/early return, ternary for either-or, && for show-or-nothing.

9. Handling Events

Attach functions to user actions. Use camelCase names and pass a **function** (not a function call!).

```
function Demo() {
  function handleClick() { alert("Clicked!"); }
  function handleChange(e) { console.log(e.target.value); }

  return (
    <div>
      <button onClick={handleClick}>Click</button>          {/* correct */}
      <button onClick={() => alert("Hi")}>Inline</button>   {/* correct */}
      {/* <button onClick={handleClick()}> WRONG - runs immediately */}
      <input onChange={handleChange} />
      <form onSubmit={(e) => { e.preventDefault(); alert("Sent"); }}>
        <button type="submit">Send</button>
      </form>
    </div>
  );
}
// Common: onClick onChange onSubmit onKeyDown onMouseEnter onFocus
```

REMEMBER: onClick={fn} passes the function. onClick={fn()} runs it too early.

10. Styling in React

Four popular ways to dress your components.

```
// 1. Normal CSS file
import "./App.css";                       // then use className="card"

// 2. Inline style (object, camelCase)
<div style={{ backgroundColor: "navy", padding: 10 }}>Hi</div>

// 3. CSS Modules (styles are private to this file)
import styles from "./Button.module.css";
<button className={styles.primary}>Go</button>

// 4. Utility classes with Tailwind CSS
<button className="bg-blue-500 text-white px-4 py-2 rounded">Go</button>

// Conditional classes
<div className={`box ${isActive ? "active" : ""}`}>Box</div>
```

REMEMBER: CSS file for global, CSS Modules for private, Tailwind for speed.

PART 2 - STATE AND HOOKS (THE HEART OF REACT)

11. useState: Giving a Component Memory

State is the component's **memory**, like a whiteboard. Change the whiteboard with the setter function and React **re-draws** the screen. Normal variables are forgotten on every render - state is remembered.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);    // [value, setter] = initial value

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(c => c - 1)}>-1</button>  {/* safe form */}
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
// Never do: count = count + 1;   (React will not notice!)
```

REMEMBER: const [value, setValue] = useState(start). Call setValue to update and redraw.

12. State Rules: Never Mutate, Always Replace

React only notices when you hand it a **new** object or array, like giving the teacher a fresh answer sheet instead of scribbling on the old one. Use spread, map and filter to make copies.

```
const [user, setUser] = useState({ name: "Asha", age: 20 });
const [todos, setTodos] = useState(["Learn React"]);

// Objects: copy, then override
setUser({ ...user, age: 21 });             // NOT user.age = 21

// Arrays
setTodos([...todos, "Build app"]);                    // add
setTodos(todos.filter(t => t !== "Learn React"));     // remove
setTodos(todos.map(t => t === "x" ? "y" : t));        // update one

// Updating from the previous value (batching-safe)
setCount(prev => prev + 1);

// State updates are asynchronous: the new value shows on the NEXT render
// setCount(5); console.log(count);   // still the old value here
```

REMEMBER: Replace, don't mutate: spread for objects, map/filter for arrays.

13. Lifting State Up

When two siblings need the same data, move the state to their **closest common parent** and pass it down as props. Like keeping the scoreboard in the middle of the stadium where everyone can see.

```
function Parent() {
  const [text, setText] = useState("");        // state lives HERE

  return (
    <>
      <Input value={text} onChange={setText} />
      <Preview value={text} />
    </>
  );
}

function Input({ value, onChange }) {
  return <input value={value} onChange={e => onChange(e.target.value)} />;
}

function Preview({ value }) {
  return <p>You typed: {value}</p>;
}
```

REMEMBER: Shared data lives in the parent. Data goes down, events go up.

14. Controlled Inputs and Forms

A **controlled input** gets its value from state, so React is always in charge, like a teacher holding the class register.

```
function SignUp() {
  const [form, setForm] = useState({ name: "", email: "", agree: false });

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm({ ...form, [name]: type === "checkbox" ? checked : value });
  }

  function handleSubmit(e) {
    e.preventDefault();                        // stop page reload
    console.log("Submitted:", form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="name" value={form.name} onChange={handleChange} />
      <input name="email" value={form.email} onChange={handleChange} />
      <input name="agree" type="checkbox" checked={form.agree}
             onChange={handleChange} />
      <button disabled={!form.agree}>Join</button>
    </form>
  );
}
```

REMEMBER: value={state} + onChange={update} = controlled input. One handler with [name].

15. useEffect: Doing Things After Render

Effects are for **side-effects**: things outside rendering such as fetching data, timers, changing the page title. The **dependency array** decides when to run: [] = once, [x] = when x changes, none = every render.

```
import { useState, useEffect } from "react";

function Demo() {
  const [count, setCount] = useState(0);

  useEffect(() => {                        // runs once after first render
    console.log("Component appeared");
  }, []);

  useEffect(() => {                        // runs when count changes
    document.title = `Count: ${count}`;
  }, [count]);

  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
// Think: "After render, if these values changed, do this."
```

REMEMBER: useEffect(fn, [deps]): [] once, [x] when x changes.

16. useEffect Cleanup

If an effect starts something (timer, listener, connection), **return a cleanup function** to switch it off - like turning off the tap when you leave.

```
function Clock() {
  const [time, setTime] = useState(new Date().toLocaleTimeString());

  useEffect(() => {
    const id = setInterval(() => {
      setTime(new Date().toLocaleTimeString());
    }, 1000);

    return () => clearInterval(id);       // cleanup when component leaves
  }, []);

  return <h2>{time}</h2>;
}

function WindowSize() {
  const [w, setW] = useState(window.innerWidth);
  useEffect(() => {
    const onResize = () => setW(window.innerWidth);
    window.addEventListener("resize", onResize);
    return () => window.removeEventListener("resize", onResize);
  }, []);
  return <p>Width: {w}</p>;
}
```

REMEMBER: Start something in an effect? Return a function that stops it.

17. useRef: A Box That Does Not Redraw

useRef is a **sticky note** that survives re-renders, but changing it does NOT redraw the screen. Two main uses: grab a DOM element, or keep a value (timer id, previous value).

```
import { useRef, useEffect } from "react";

function FocusInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus();             // touch the real DOM element
  }, []);

  return <input ref={inputRef} placeholder="I get focus automatically" />;
}

function Stopwatch() {
  const timerId = useRef(null);           // remembers value, no re-render
  const start = () => { timerId.current = setInterval(() => console.log("tick"), 1000); };
  const stop  = () => clearInterval(timerId.current);
  return <><button onClick={start}>Start</button><button onClick={stop}>Stop</button></>;
}
```

REMEMBER: useRef = DOM access or a silent memory. ref.current holds the value.

18. useContext: Skipping the Prop Chain

Passing props through 5 levels is like passing a note through 5 people. **Context** is a **loudspeaker**: any component below can listen directly. Great for theme, logged-in user, language.

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext("light");          // 1. create

function ThemeProvider({ children }) {                // 2. provide
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

function ThemeButton() {                              // 3. use anywhere below
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
      Now: {theme}
    </button>
  );
}
// Wrap the app:  <ThemeProvider><App /></ThemeProvider>
```

REMEMBER: createContext -> Provider value -> useContext. No more prop drilling.

19. useReducer: State with Rules

When state has many actions, use a **reducer**: a manager that takes the current state plus an **action** (a request slip) and returns the new state. Like a bank teller handling deposit/withdraw slips.

```
import { useReducer } from "react";

function reducer(state, action) {
  switch (action.type) {
    case "add":    return [...state, { id: Date.now(), text: action.text }];
    case "remove": return state.filter(t => t.id !== action.id);
    case "clear":  return [];
    default:       return state;
  }
}

function Todos() {
  const [todos, dispatch] = useReducer(reducer, []);
  return (
    <>
      <button onClick={() => dispatch({ type: "add", text: "Study" })}>Add</button>
      <button onClick={() => dispatch({ type: "clear" })}>Clear</button>
      {todos.map(t => (
        <p key={t.id} onClick={() => dispatch({ type: "remove", id: t.id })}>
          {t.text}
        </p>
      ))}
    </>
  );
}
```

REMEMBER: dispatch(action) -> reducer(state, action) -> new state.

20. useMemo: Remember an Expensive Result

Like a **notebook for slow calculations**: only recalculate when the inputs change.

```
import { useMemo, useState } from "react";

function Search({ products }) {
  const [query, setQuery] = useState("");
  const [dark, setDark] = useState(false);

  const filtered = useMemo(() => {
    console.log("filtering...");                // runs only when needed
    return products.filter(p =>
      p.name.toLowerCase().includes(query.toLowerCase())
    );
  }, [products, query]);                        // recompute when these change

  return (
    <div className={dark ? "dark" : ""}>
      <button onClick={() => setDark(!dark)}>Toggle theme</button>   {/* no re-filter */}
      <input value={query} onChange={e => setQuery(e.target.value)} />
      {filtered.map(p => <p key={p.id}>{p.name}</p>)}
    </div>
  );
}
// Use only for truly slow work. Don't wrap everything.
```

REMEMBER: useMemo(() => value, [deps]) caches a calculated value.

21. useCallback and React.memo

React.memo says 'skip re-drawing this child if props are the same'. But a function made inside a parent is **new on every render**, so props look different. **useCallback** keeps the same function, so memo works.

```
import { memo, useCallback, useState } from "react";

const Child = memo(function Child({ onClick }) {
  console.log("Child rendered");
  return <button onClick={onClick}>Child button</button>;
});

function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log("clicked");
  }, []);                                   // same function every time

  return (
    <>
      <button onClick={() => setCount(count + 1)}>Parent: {count}</button>
      <Child onClick={handleClick} />      {/* does not re-render on count */}
    </>
  );
}
```

REMEMBER: memo skips unchanged children. useCallback keeps function props stable.

22. Custom Hooks: Your Own Reusable Logic

A custom hook is a **function starting with 'use'** that bundles hooks together - like saving a favourite recipe so you never rewrite it.

```
import { useState, useEffect } from "react";

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initial;
  });
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);
  return [value, setValue];
}

function useToggle(start = false) {
  const [on, setOn] = useState(start);
  return [on, () => setOn(o => !o)];
}

// Use them
function Settings() {
  const [name, setName] = useLocalStorage("name", "");
  const [open, toggle] = useToggle();
  return <input value={name} onChange={e => setName(e.target.value)} />;
}
```

REMEMBER: Same logic in 2 places? Move it into a useSomething function.

23. Rules of Hooks and Other Useful Hooks

Two rules: (1) only call hooks at the **top level** (not inside if/loops), (2) only inside components or custom hooks. React remembers hooks by their **order**.

```
function Bad({ show }) {
  // if (show) { const [x, setX] = useState(0); }   // WRONG: inside if
  const [x, setX] = useState(0);                     // RIGHT: top level
  return <p>{x}</p>;
}

// Other handy hooks
// useId()               unique id for label/input pairs
// useTransition()       mark an update as low-priority (keeps typing smooth)
// useDeferredValue(v)   a delayed copy of v for heavy lists
// useLayoutEffect       like useEffect, but runs BEFORE the screen paints
// useImperativeHandle   choose what a parent can call on your ref

const id = useId();
// <label htmlFor={id}>Name</label><input id={id} />
```

REMEMBER: Hooks at the top level only, in the same order every render.

PART 3 - BUILDING REAL APPS

24. Fetching Data from an API

Fetch data **after** the component appears (useEffect), and always handle three moods: **loading, error, success**.

```
import { useState, useEffect } from "react";

function Users() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let ignore = false;                          // avoid stale updates
    async function load() {
      try {
        const res = await fetch("https://jsonplaceholder.typicode.com/users");
        if (!res.ok) throw new Error("Server error " + res.status);
        const data = await res.json();
        if (!ignore) setUsers(data);
      } catch (err) {
        if (!ignore) setError(err.message);
      } finally {
        if (!ignore) setLoading(false);
      }
    }
    load();
    return () => { ignore = true; };
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Oops: {error}</p>;
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

REMEMBER: Fetch in useEffect. Handle loading, error, success. Libraries like TanStack Query do this for you.

25. React Router: Multiple Pages

React makes ONE page, so **React Router** swaps components based on the URL, like changing the channel without changing the TV. Install: **npm install react-router-dom**.

```
import { BrowserRouter, Routes, Route, Link, NavLink } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <NavLink to="/about">About</NavLink>   {/* knows when it is active */}
      </nav>

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<h1>404 Not Found</h1>} />
      </Routes>
    </BrowserRouter>
  );
}
// Use <Link>, not <a>, so the page does not reload.
```

REMEMBER: BrowserRouter > Routes > Route path/element. Link to move around.

26. Router: Params, Navigation, Nested and Protected Routes

URL parts become variables (**:id**), you can redirect in code, and you can guard private pages.

```
import { useParams, useNavigate, Navigate, Outlet } from "react-router-dom";

// <Route path="/user/:id" element={<User />} />
function User() {
  const { id } = useParams();                  // from the URL
  const navigate = useNavigate();
  return (
    <>
      <h1>User {id}</h1>
      <button onClick={() => navigate("/")}>Go home</button>
      <button onClick={() => navigate(-1)}>Back</button>
    </>
  );
}

function PrivateRoute({ isLoggedIn }) {        // guard
  return isLoggedIn ? <Outlet /> : <Navigate to="/login" replace />;
}

// Nested routes: parent renders <Outlet /> where the child appears
// <Route element={<PrivateRoute isLoggedIn={ok} />}>
//   <Route path="/dashboard" element={<Dashboard />} />
// </Route>
```

REMEMBER: useParams reads the URL, useNavigate moves, Outlet shows child routes.

27. Global State: Redux Toolkit and Zustand

When many far-apart components share data (cart, user), a **global store** is a central warehouse. **Zustand** is the lightest; **Redux Toolkit** is the industry standard.

```
// ---- Zustand: npm install zustand ----
import { create } from "zustand";

const useCart = create((set) => ({
  items: [],
  add: (item) => set((s) => ({ items: [...s.items, item] })),
  clear: () => set({ items: [] }),
}));

function CartButton() {
  const count = useCart((s) => s.items.length);   // subscribe to a slice
  const add = useCart((s) => s.add);
  return <button onClick={() => add("Shoes")}>Cart ({count})</button>;
}

// ---- Redux Toolkit: npm install @reduxjs/toolkit react-redux ----
import { createSlice, configureStore } from "@reduxjs/toolkit";
const counter = createSlice({
  name: "counter", initialState: { value: 0 },
  reducers: { inc: (s) => { s.value += 1; } },     // "mutating" is safe here
});
export const store = configureStore({ reducer: { counter: counter.reducer } });
// <Provider store={store}>  useSelector(s => s.counter.value)  useDispatch()
```

REMEMBER: Small app: useState/Context. Big shared state: Zustand or Redux Toolkit.

28. Forms with React Hook Form

For bigger forms, **React Hook Form** handles values, validation and errors with very little code. Install: npm install react-hook-form.

```
import { useForm } from "react-hook-form";

function Login() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  const onSubmit = (data) => console.log(data);   // { email, password }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input placeholder="Email"
        {...register("email", { required: "Email is required" })} />
      {errors.email && <p>{errors.email.message}</p>}

      <input type="password" placeholder="Password"
        {...register("password", {
          required: "Password is required",
          minLength: { value: 8, message: "Minimum 8 characters" },
        })} />
      {errors.password && <p>{errors.password.message}</p>}

      <button>Login</button>
    </form>
  );
}
```

REMEMBER: register fields, handleSubmit validates, errors show messages.

29. Error Boundaries

A **safety net** for the UI: if a child component crashes, show a friendly message instead of a blank white page. It must be a class (or use the react-error-boundary package).

```
import { Component } from "react";

class ErrorBoundary extends Component {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };               // switch to fallback
  }
  componentDidCatch(error, info) {
    console.log("Logged:", error, info);     // send to a log service
  }
  render() {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}

// Usage
// <ErrorBoundary><Dashboard /></ErrorBoundary>

// Easier: npm install react-error-boundary
// <ErrorBoundary fallback={<p>Oops</p>}> ... </ErrorBoundary>
```

REMEMBER: Wrap risky parts in an ErrorBoundary. It catches render errors, not event-handler errors.

30. Portals and Modals

A **portal** renders a child somewhere else in the page (like a pop-up that teleports to the top of the building) while still belonging to its parent. Perfect for modals and tooltips.

```
import { createPortal } from "react-dom";
import { useState } from "react";

function Modal({ children, onClose }) {
  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div className="modal" onClick={(e) => e.stopPropagation()}>
        {children}
        <button onClick={onClose}>Close</button>
      </div>
    </div>,
    document.body                       // teleports here
  );
}

function App() {
  const [open, setOpen] = useState(false);
  return (
    <>
      <button onClick={() => setOpen(true)}>Open</button>
      {open && <Modal onClose={() => setOpen(false)}>Hello from modal</Modal>}
    </>
  );
}
```

REMEMBER: createPortal(jsx, domNode) = render here, show there.

31. Lazy Loading and Suspense

Don't download the whole shop to buy milk. **lazy** loads a component only when needed, and **Suspense** shows a placeholder while waiting. Makes your first page load faster.

```
import { lazy, Suspense } from "react";

const Dashboard = lazy(() => import("./Dashboard"));   // separate file chunk
const Settings  = lazy(() => import("./Settings"));

function App() {
  return (
    <Suspense fallback={<p>Loading page...</p>}>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/settings"  element={<Settings />} />
      </Routes>
    </Suspense>
  );
}
// Rule: lazy-load per page/route, and the component needs a default export.
```

REMEMBER: lazy(() => import(...)) + . Load code only when needed.

32. Patterns: Composition, HOC and Render Props

React prefers **composition** (combining small pieces) over inheritance. Older reuse patterns: **HOC** = a function that wraps a component and gives it extra powers; **render props** = pass a function that says what to draw. Today, custom hooks replace most of these.

```
// Composition: slots filled with other components
function Layout({ header, sidebar, children }) {
  return (<div><header>{header}</header><aside>{sidebar}</aside><main>{children}</main></div>);
}
// <Layout header={<Nav />} sidebar={<Menu />}><Feed /></Layout>

// Higher-Order Component
function withLoading(Component) {
  return function Wrapped({ loading, ...props }) {
    return loading ? <p>Loading...</p> : <Component {...props} />;
  };
}
const UserListWithLoading = withLoading(UserList);

// Render prop
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY })}>
    {render(pos)}
  </div>;
}
// <MouseTracker render={p => <p>{p.x}, {p.y}</p>} />
```

REMEMBER: Prefer composition + custom hooks. HOC and render props are older tricks.

33. Class Components (Legacy)

Before hooks, components were **classes**. You will still see them in old projects, so know the translation. New code: use function components.

```
import { Component } from "react";

class Counter extends Component {
  state = { count: 0 };

  componentDidMount() { console.log("mounted"); }          // = useEffect(..., [])
  componentDidUpdate() { console.log("updated"); }         // = useEffect(...)
  componentWillUnmount() { console.log("bye"); }           // = effect cleanup

  increment = () => this.setState({ count: this.state.count + 1 });

  render() {
    return <button onClick={this.increment}>{this.state.count}</button>;
  }
}

// Lifecycle story: Mount (born) -> Update (grows) -> Unmount (leaves)
```

REMEMBER: Lifecycle: mount, update, unmount. Hooks do the same job in function components.

PART 4 - MODERN AND ADVANCED REACT

34. Performance: Why Components Re-render

A component re-renders when its **state changes, its props change, or its parent re-renders**. Usually that is fast. Only optimize when you SEE slowness (React DevTools Profiler).

```
// 1. Move state down, closer to where it is used
function Page() {
  return (<><SlowChart /><SearchBox /></>);   // typing in SearchBox no longer
}                                             // re-renders SlowChart
function SearchBox() { const [q, setQ] = useState(""); /* ... */ }

// 2. Pass children so parts do not re-render
// 3. memo / useMemo / useCallback for proven slow spots
// 4. Stable keys in lists, never random keys
// 5. Virtualize very long lists (react-window): draw only visible rows
// 6. Lazy-load pages (React.lazy)
// 7. Debounce typing for search boxes

// React Compiler (modern React) can add much of this memoization for you.
```

REMEMBER: Measure first. Move state down before reaching for memo.

35. React 19 Features

React 19 (stable since late 2024) made forms and async work simpler: **Actions**, **useActionState**, **useOptimistic**, the **use** API, **ref as a normal prop**, and built-in <title>/<meta> support. Check react.dev for the newest details.

```
import { useActionState, useOptimistic, use } from "react";

// Form Action: pass a function to <form action>
function Subscribe() {
  const [message, formAction, isPending] = useActionState(
    async (prev, formData) => {
      const email = formData.get("email");
      await fakeApi(email);
      return "Subscribed: " + email;
    },
    ""                                    // initial state
  );
  return (
    <form action={formAction}>
      <input name="email" />
      <button disabled={isPending}>{isPending ? "..." : "Join"}</button>
      <p>{message}</p>
    </form>
  );
}

// useOptimistic: show the result instantly, fix it if the server fails
// const [optimisticList, addOptimistic] = useOptimistic(list, (s, n) => [...s, n]);

// use(): read a Promise or Context inside render
// const user = use(userPromise);

// ref is now a normal prop (no forwardRef needed):
// function Input({ ref, ...props }) { return <input ref={ref} {...props} />; }
```

REMEMBER: React 19: form actions, useActionState, useOptimistic, use(), ref as prop.

36. Server Components and Next.js

Normal React runs in the browser. **Server Components** run on the server and send ready HTML - less JavaScript for the user, direct database access. **Next.js** is the most popular framework that gives you this plus routing, SEO and deployment.

```
// Next.js (App Router)   npx create-next-app@latest

// app/page.jsx  -> the "/" page. Server Component by default
export default async function Home() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts");
  const posts = await res.json();            // runs on the SERVER
  return <ul>{posts.slice(0, 5).map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}

// app/components/Like.jsx -> needs state/clicks, so mark as Client Component
"use client";
import { useState } from "react";
export default function Like() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n + 1)}>Like {n}</button>;
}

// Folder = route:  app/about/page.jsx -> /about
//                  app/blog/[id]/page.jsx -> /blog/5
```

REMEMBER: Server Component = no useState, fast. 'use client' = interactive. Next.js = full toolkit.

37. TypeScript with React

TypeScript adds **type checking**: a spell-checker for your code. It catches wrong props and typos before you run the app. Use .tsx files.

```
import { useState, ReactNode } from "react";

type ButtonProps = {
  label: string;
  onClick: () => void;
  disabled?: boolean;               // ? = optional
  children?: ReactNode;
};

function Button({ label, onClick, disabled = false }: ButtonProps) {
  return <button onClick={onClick} disabled={disabled}>{label}</button>;
}

type User = { id: number; name: string };
const [users, setUsers] = useState<User[]>([]);       // generic state type
const [user, setUser] = useState<User | null>(null);

function handle(e: React.ChangeEvent<HTMLInputElement>) {
  console.log(e.target.value);
}
// <Button label="Go" onClick={() => {}} />   <Button />  -> red underline!
```

REMEMBER: type Props = {...}; function Comp(props: Props). Typos become red underlines.

38. Testing React Components

Testing = a **robot user** that checks your app works. Popular combo: **Vitest + React Testing Library**. Test what the user sees and does, not the internals.

```
// npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
import { render, screen, fireEvent } from "@testing-library/react";
import { describe, it, expect } from "vitest";
import Counter from "./Counter";

describe("Counter", () => {
  it("starts at 0", () => {
    render(<Counter />);
    expect(screen.getByText("Count: 0")).toBeInTheDocument();
  });

  it("increases when clicked", () => {
    render(<Counter />);
    fireEvent.click(screen.getByRole("button", { name: "+1" }));
    expect(screen.getByText("Count: 1")).toBeInTheDocument();
  });
});
// Run:  npx vitest
```

REMEMBER: render -> find like a user -> act -> expect.

39. Build and Deploy

Developing and publishing are different. **Build** makes a small, fast, optimized folder you upload to the internet.

```
# 1. Create the production build
npm run build            # makes the dist/ folder (Vite)
npm run preview          # test the build locally

# 2. Deploy (free options)
#   Vercel / Netlify : connect your GitHub repo -> auto deploy on every push
#   GitHub Pages     : upload dist/
#   Firebase Hosting : firebase deploy

# Environment variables (Vite): .env file, names must start with VITE_
# VITE_API_URL=https://api.example.com
# const url = import.meta.env.VITE_API_URL;

# Single-page apps need a rewrite rule so /about works on refresh
# (Netlify: _redirects file with   /*  /index.html  200)
```

REMEMBER: npm run build -> upload dist/ (or connect GitHub to Vercel/Netlify).

40. Folder Structure and Best Practices

Keep the project tidy like a well-organized kitchen.

```
src/
  components/      small reusable pieces (Button, Card, Modal)
  pages/           one file per screen (Home, Login, Dashboard)
  hooks/           custom hooks (useLocalStorage.js)
  context/         ThemeContext, AuthContext
  services/        API calls (api.js)
  utils/           helper functions
  assets/          images, fonts
  App.jsx  main.jsx

// Best practices
// 1. One component = one job; split when a file passes ~150 lines
// 2. Keep state as LOW as possible, lift only when needed
// 3. Never mutate state; derive values instead of storing them
//      const total = items.reduce(...)   // not useState for total
// 4. Unique stable keys in lists
// 5. Handle loading and error states
// 6. Extract repeated logic into custom hooks
// 7. Use ESLint + Prettier; name components in PascalCase
```

REMEMBER: Small components, low state, derived values, custom hooks.

41. Common Mistakes to Avoid

Every React beginner meets these. Spot them early!

```
// 1. Mutating state
//    items.push(x); setItems(items);          WRONG
//    setItems([...items, x]);                 RIGHT

// 2. Infinite loop: setState inside useEffect with no deps
//    useEffect(() => { setCount(count + 1); });          // loops forever!
//    useEffect(() => { setCount(c => c + 1); }, []);     // runs once

// 3. Missing dependencies
//    useEffect(() => { fetchUser(id); }, []);            // stale id!
//    useEffect(() => { fetchUser(id); }, [id]);          // correct

// 4. Calling the function instead of passing it
//    <button onClick={doIt()}>      WRONG
//    <button onClick={doIt}>        RIGHT

// 5. Index as key in changing lists
// 6. Storing derived data in state
// 7. Forgetting cleanup for timers and listeners
// 8. Defining a component INSIDE another component (it resets every render)
// 9. Reading state right after setState (it is not updated yet)
```

REMEMBER: Replace state, add deps, pass functions, clean up effects.

FINAL PROJECT - To-Do App (components, props, state, reducer, custom hook)

Create a Vite React project and put these files in **src/**. It uses almost everything from the notes.

```
// ---------- src/useLocalStorage.js ----------
import { useState, useEffect } from "react";
export function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initial;
  });
  useEffect(() => { localStorage.setItem(key, JSON.stringify(value)); }, [key, value]);
  return [value, setValue];
}

// ---------- src/TodoItem.jsx ----------
export default function TodoItem({ todo, onToggle, onDelete }) {
  return (
    <li>
      <span onClick={() => onToggle(todo.id)}
            style={{ textDecoration: todo.done ? "line-through" : "none", cursor: "pointer" }}>
        {todo.text}
      </span>
      <button onClick={() => onDelete(todo.id)}>x</button>
    </li>
  );
}

// ---------- src/App.jsx ----------
import { useState } from "react";
import { useLocalStorage } from "./useLocalStorage";
import TodoItem from "./TodoItem";

export default function App() {
  const [todos, setTodos] = useLocalStorage("todos", []);
  const [text, setText] = useState("");
  const [filter, setFilter] = useState("all");

  function addTodo(e) {
    e.preventDefault();
    if (!text.trim()) return;
    setTodos([...todos, { id: Date.now(), text: text.trim(), done: false }]);
    setText("");
  }
  const toggle = (id) => setTodos(todos.map(t => t.id === id ? { ...t, done: !t.done } : t));
  const remove = (id) => setTodos(todos.filter(t => t.id !== id));

  const visible = todos.filter(t =>
    filter === "all" ? true : filter === "done" ? t.done : !t.done);   // derived, not state

  return (
    <div>
      <h1>My To-Do List</h1>
      <form onSubmit={addTodo}>
        <input value={text} onChange={e => setText(e.target.value)} placeholder="New task" />
        <button>Add</button>
      </form>
      {["all", "active", "done"].map(f => (
        <button key={f} onClick={() => setFilter(f)} disabled={filter === f}>{f}</button>
      ))}
      <ul>
        {visible.map(t => <TodoItem key={t.id} todo={t} onToggle={toggle} onDelete={remove} />)}
      </ul>
      <p>{todos.filter(t => !t.done).length} tasks left</p>
    </div>
  );
}
```

ONE-PAGE CHEAT SHEET

```
SETUP       npm create vite@latest app -- --template react   |   npm run dev
JSX         one parent | className | htmlFor | close all tags | { js } | style={{ }}
COMPONENT   function Name({ prop1, prop2 = x, children }) { return <div/>; }
LIST        items.map(i => <Item key={i.id} {...i} />)
CONDITION   cond ? <A/> : <B/>      cond && <A/>      if (x) return <Loading/>
EVENT       onClick={fn}  onChange={e => set(e.target.value)}  onSubmit + preventDefault
STATE       const [v, setV] = useState(0)     setV(prev => prev + 1)
            objects: setU({...u, a: 1})   arrays: [...a, x]  a.filter()  a.map()
EFFECT      useEffect(() => { ...; return cleanup; }, [deps])   [] once, [x] on change
REF         const r = useRef(null);  <input ref={r}/>  r.current.focus()
CONTEXT     createContext() -> <Ctx.Provider value> -> useContext(Ctx)
REDUCER     const [s, dispatch] = useReducer(reducer, init);  dispatch({type:"add"})
PERFORMANCE useMemo(() => val, [deps])  useCallback(fn, [deps])  memo(Component)
CUSTOM HOOK function useThing() { ...hooks...; return [a, b]; }   name starts with use
ROUTER      BrowserRouter > Routes > Route path element | Link | useParams | useNavigate
FETCH       useEffect + fetch + loading / error / data states  (or TanStack Query)
LAZY        const P = lazy(() => import("./P"));  <Suspense fallback={...}>
PORTAL      createPortal(jsx, document.body)
REACT 19    <form action={fn}>  useActionState  useOptimistic  use()  ref as prop
RULES       hooks at top level only | never mutate state | props are read-only
            data flows DOWN (props), events flow UP (callbacks)
```

Your 90-Day React Roadmap

Days 1-10	Part 1: JSX, components, props, lists, events. Build a profile card and product list.
Days 11-30	Part 2: useState, useEffect, forms, useRef. Build a counter, to-do app, and weather app with fetch.
Days 31-50	useContext, useReducer, custom hooks, React Router. Build a multi-page app with login and protected routes.
Days 51-70	Part 3: global state (Zustand/Redux Toolkit), forms library, lazy loading. Build a shopping cart.
Days 71-90	Part 4: TypeScript, testing, Next.js. Build and deploy one real project to Vercel for your portfolio.

BIG IDEA: UI = function(state). Change the state, React changes the screen. Everything else is detail.